

Kubernetes

Explained Simply

**The concepts and commands
you'll actually use.**

Core ideas in one sentence each, 40+ everyday kubectl commands, and what to do when a pod won't start.

 15 core concepts

 Cluster map

 Context

 Looking around

 Deploying

 Troubleshooting

 Config & secrets

 Shortcuts

 Pod statuses

 YAML explained

by

Subhash Sharma

Swipe →

What is Kubernetes?

One sentence, one picture, one kitchen



KUBERNETES IN ONE SENTENCE

Open-source software that runs your containerised apps across many machines, and keeps them running the way you asked.

CLUSTER

Control plane the brain: decides what runs where

API server

Scheduler

Controllers

etcd

Worker node 1

pod

pod

kubelet + runtime

Worker node 2

pod

pod

pod

kubelet + runtime

Worker node 3

pod

pod

kubelet + runtime

CONTROL PLANE

The head chef. Takes orders and decides which station cooks what.

WORKER NODE

A cooking station. The machine that does the actual work.

POD

A dish being made. Your app's container(s), running.

DESIRED STATE

The order ticket. Kubernetes keeps fixing things until reality matches it.



The building blocks

The words you'll hear in every Kubernetes chat

Cluster THE WHOLE THING

A set of machines that Kubernetes manages as one.

Think of it as: the whole restaurant: kitchen, staff and a manager keeping it all running.

Node A MACHINE

One machine (virtual or physical) that runs your pods.

Think of it as: one cooking station. If a station breaks, the work moves to another.

Pod SMALLEST UNIT

Wraps one or more containers that share an IP and storage.

Think of it as: a lunchbox: usually one main item, sometimes a small side (a 'sidecar'). Pods are disposable.

Container PACKAGED APP

Your app plus everything it needs, packed into an image.

Think of it as: a ready meal: same result wherever you heat it. Run by a runtime such as containerd.

Namespace A FOLDER

A named area that keeps groups of resources apart.

Think of it as: folders on a shared drive: dev, test and prod can reuse names without clashing.



Running your app

Objects that start, copy and schedule your pods

Deployment STATELESS APPS

Keeps several identical pods running and rolls out updates.

Think of it as: a shift manager: 'I need 3 cooks on at all times.' One leaves, a replacement starts.

ReplicaSet COPY KEEPER

Makes sure the right number of pod copies exist.

Think of it as: the headcount clipboard. Deployments create these for you, so you rarely touch them.

StatefulSet STATEFUL APPS

Like a Deployment, but each pod keeps its name and disk.

Use it when: running databases or queues, where pod db-0 must stay db-0, with its own storage.

DaemonSet ONE PER NODE

Runs one copy of a pod on every node (or chosen nodes).

Think of it as: a smoke alarm in every room. Used for log collectors and monitoring agents.

Job & CronJob RUN TO FINISH

Job: run a task until it's done. CronJob: on a schedule.

Think of it as: a one-off deep clean versus the nightly 2am tidy-up. Great for backups and reports.



Networking, config & storage

How pods are reached, configured and given disks

Service STABLE ADDRESS

A fixed name and IP that spreads traffic across pods.

Think of it as: a restaurant's phone number: staff change, the number doesn't. Pods come and go; the Service stays.

Ingress WEB ENTRY

Routes outside web traffic to Services by host or path.

Think of it as: a receptionist directing visitors. Needs an Ingress controller; Gateway API is the newer option.

ConfigMap SETTINGS

Stores non-secret settings outside your container image.

Think of it as: a note on the fridge door: change it without rebuilding the app.

Secret SENSITIVE DATA

Holds passwords, tokens and keys for your pods to use.

Watch out: by default it's only base64-encoded, not encrypted. Lock down who can read it.

Volume & PVC STORAGE

Volumes give pods disk space; a PVC requests lasting storage.

Think of it as: a request form: 'I need 10 GB'. The cluster finds a disk that outlives any single pod.



Getting started & context

Which cluster am I talking to?

kubectl version

Shows your kubectl version and the cluster's version.

kubectl cluster-info

Checks you can reach the cluster and shows its address.

kubectl config get-contexts

Lists the clusters in your kubeconfig. * marks the current one.

kubectl config current-context

Shows which cluster you're pointed at right now.

kubectl config use-context <name>

Switches cluster. Double-check before deleting anything!

kubectl config set-context --current --namespace=dev

Makes dev your default namespace, so you can skip -n dev.

kubectl get namespaces

Lists every namespace in the cluster.

Good to know: your settings live in ~/.kube/config (the 'kubeconfig'). Add `-n <namespace>` to any command to target another namespace.



Looking around

See what's running, and what's wrong

```
kubectl get pods
```

Pods in your current namespace, with status and restarts.

```
kubectl get pods -A
```

Pods in all namespaces (-A is short for --all-namespaces).

```
kubectl get pods -o wide
```

Adds each pod's IP address and the node it's on.

```
kubectl get all
```

Pods, Services, Deployments and more (not ConfigMaps or Secrets).

```
kubectl get nodes
```

Lists the machines and whether each one is Ready.

```
kubectl describe pod <pod>
```

Full details. Read the Events section at the bottom first.

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

Recent events in the namespace, newest at the bottom.

```
kubectl events --types=Warning
```

Just the warnings: the quickest way to spot trouble.



Deploying & updating

Ship it, update it, roll it back

```
kubectl apply -f app.yaml
```

Creates or updates everything in the file. The everyday way.

```
kubectl create deployment web --image=nginx:1.28
```

Quick Deployment without writing YAML. Container is named nginx.

```
kubectl set image deployment/web nginx=nginx:1.29
```

Changes the nginx container's image and starts a rolling update.

```
kubectl rollout status deployment/web
```

Watches the update until it finishes (or gets stuck).

```
kubectl rollout history deployment/web
```

Lists previous versions (revisions) of the Deployment.

```
kubectl rollout undo deployment/web
```

Goes back to the previous revision. Your undo button.

```
kubectl scale deployment/web --replicas=5
```

Changes the number of running copies to 5.

```
kubectl delete -f app.yaml
```

Deletes everything in the file. There's no 'are you sure?'



Troubleshooting

When something isn't right

```
kubectl logs <pod>
```

Prints the pod's log output.

```
kubectl logs -f <pod>
```

Follows the logs live, like tail -f. Ctrl+C to stop.

```
kubectl logs <pod> --previous
```

Logs from the last run that crashed. Vital for restarts.

```
kubectl logs <pod> -c <container>
```

Picks one container when a pod has several.

```
kubectl exec -it <pod> -- sh
```

Opens a shell inside. No shell in the image? Try kubectl debug.

```
kubectl describe pod <pod>
```

Shows events, restarts, limits and why the last run ended.

```
kubectl top pods
```

```
kubectl top nodes
```

Live CPU and memory use. Needs Metrics Server installed.

```
kubectl port-forward svc/web 8080:80
```

Opens localhost:8080 on your machine to port 80 of the web Service.



Config & secrets

Keep settings and passwords out of your image

```
kubectl create configmap app-config \
  --from-literal=MODE=prod
```

Creates a ConfigMap holding one setting: MODE=prod.

```
kubectl get configmap app-config -o yaml
```

Shows what's stored in it.

```
kubectl set env deployment/web \
  --from=configmap/app-config
```

Loads every key into the app as environment variables.

```
kubectl create secret generic db-creds \
  --from-literal=password=changeme
```

Creates a Secret. Typed values end up in your shell history.

```
kubectl get secret db-creds -o yaml
```

The value looks scrambled, but it's only base64.

```
kubectl get secret db-creds \
  -o jsonpath='{.data.password}' | base64 -d
```

Decodes it instantly. Proof that base64 isn't encryption.

Secrets are encoded, not encrypted. Anyone who can read them can decode them. Limit access with RBAC, turn on encryption at rest, or use an external secrets manager.



Handy shortcuts

Type less, learn faster

```
alias k=kubectl
```

Now `k get pods` works. Add it to `~/.bashrc` or `~/.zshrc`.

```
kubectl get po,svc,deploy,ns
```

Short names: pods, services, deployments, namespaces.

```
kubectl api-resources
```

Lists every resource type, with its short name.

```
kubectl get deploy web -o yaml
```

Shows the full live YAML of any object.

```
kubectl create deployment web --image=nginx:1.28 \  
--dry-run=client -o yaml > web.yaml
```

Writes starter YAML without creating anything. Edit, then apply.

```
kubectl explain deployment.spec.replicas
```

Built-in documentation for any field.

```
kubectl get pods -l app=web
```

Filters by label: only pods tagged `app=web`.

```
kubectl get pods -w
```

Watches for changes live. Ctrl+C to stop.

Pod won't start?



Run `kubectl get pods` and read the STATUS column

CrashLoopBackOff The app keeps crashing

It starts, crashes and restarts, with longer pauses each time. Often a bug or bad config.

TRY FIRST `kubectl logs <pod> --previous`

ImagePullBackOff / ErrImagePull Can't get the image

Wrong image name or tag, or a private registry without login.

TRY FIRST `kubectl describe pod <pod>`

Pending Waiting for a node

No node can take it yet: not enough CPU or memory, or its storage isn't ready.

TRY FIRST `kubectl describe pod <pod>`

OOMKilled Ran out of memory

Hit its memory limit and was killed. Raise it, or fix the leak.

TRY FIRST `kubectl describe pod <pod>`

CreateContainerConfigError Missing config

It refers to a ConfigMap, Secret or key that doesn't exist.

TRY FIRST `kubectl describe pod <pod>`

Your first YAML, explained

A Deployment plus a Service, in one file



web.yaml

```
apiVersion: apps/v1
kind: Deployment ❶
metadata:
  name: web
spec:
  replicas: 3 ❷
  selector:
    matchLabels:
      app: web ❸
  template: ❹
    metadata:
      labels:
        app: web ❸
    spec:
      containers:
        - name: web
          image: nginx:1.29 ❺
--- ❻
apiVersion: v1
kind: Service ❼
metadata:
  name: web
spec:
  selector:
    app: web ❽
  ports:
    - port: 80 ❾
      targetPort: 80
```

❶ kind: Deployment

What to create. It keeps your pods running.

❷ replicas: 3

Always run 3 copies. One dies, another starts.

❸ app: web

A label (name tag). The selector must match the pod labels.

❹ template

The blueprint for every pod.

❺ image

What to run. Pin a version; avoid :latest.

❻ ---

Separates two objects in one file.

❼ kind: Service

A stable name and IP in front of the pods.

❽ selector

Sends traffic to pods labelled app: web.

❾ port / targetPort

Listens on 80, forwards to port 80 in the container.

Run it:

kubectl apply -f web.yaml

Found this **useful**?



Save it

Keep it handy for your next kubectl session.



Share it

Repost to help someone learning Kubernetes.



Follow Subhash Sharma

For more plain-English tech guides.

About me: web developer from 1994 to 2016, now an application support and tech lead. I explain tech the way I wish someone had explained it to me.

Kubernetes is a registered trademark of The Linux Foundation. Not affiliated. Commands checked against the official kubectl documentation (kubernetes.io) in September 2026; always test in a safe environment first.